

Weekly Assignment — Project Management API Enhancement

1. Objective

This week, you need to enhance the **Project Management REST API** that you have already developed.

Until now, you have implemented:

1. A Node.js + Express.js API.
2. MongoDB database using Mongoose.
3. Three models/collections:
 - Users
 - Projects
 - Tasks
4. CRUD operations for all three models.
5. Authentication using **JWT (JSON Web Token)**.
6. Password security using **bcrypt hashing** instead of storing plain-text passwords.

Now, the next requirement is to improve the API responses by replacing MongoDB reference IDs with the corresponding actual data.

2. Existing Project Structure

You already have three main collections.

2.1) Users

Example:

```
{
  "_id": "68a10001",
  "name": "Akash",
  "userId": "akash01",
  "email": "akash@gmail.com",
  "role": "Developer"
```

```
}
```

2.2) Projects

A Project contains references to Users:

```
{  
  "_id": "68b20001",  
  "projectName": "EstiMate-Pro",  
  "projectId": "0001",  
  "managerId": "68a10001",  
  "ownerId": "68a10002",  
  "createdBy": "68a10001",  
  "updatedBy": "68a10003"  
}
```

Here:

managerId → User _id

ownerId → User _id

createdBy → User _id

updatedBy → User _id

2.3) Tasks

A Task contains references to both a Project and a User:

```
{  
  "_id": "68c30001",  
  "taskName": "Implement Login API",  
  "taskId": "0001",  
  "projectId": "68b20001",  
  "userId": "68a10001"  
}
```

Here:

projectId → Project_id

userId → User_id

3. Current Problem

Currently, when you retrieve a Project, the API returns only the MongoDB IDs.

For example:

Current Project Response

```
{  
  "projectName": "EstiMate-Pro",  
  "projectId": "0001",  
  "managerId": "68a10001",  
  "ownerId": "68a10002",  
  "createdBy": "68a10001",  
  "updatedBy": "68a10003"  
}
```

A person receiving this response has to make additional API calls to understand:

68a10001 → Which user?

68a10002 → Which user?

68a10003 → Which user?

This is not convenient for the client application.

4. New Requirement

You must modify the **Project and Task API responses** so that referenced MongoDB IDs are replaced with the corresponding data from their related collections.

In simple words:

If a document contains an ID of another document, retrieve that related document and include its required information in the response.

5. Project Response — Expected Result

Before

Currently:

```
{  
  "projectName": "EstiMate-Pro",  
  "projectId": "0001",  
  "managerId": "68a10001",  
  "ownerId": "68a10002",  
  "createdBy": "68a10001",  
  "updatedBy": "68a10003"  
}
```

After

The response should contain the actual User information:

```
{  
  "projectName": "EstiMate-Pro",  
  "projectId": "0001",  
  
  "managerId": {  
    "_id": "68a10001",  
    "name": "Akash",  
    "userId": "akash01",  
    "email": "akash@gmail.com",  
    "role": "Manager"  
  },  
  
  "ownerId": {  
    "_id": "68a10002",
```

```
"name": "Rupesh",
"userId": "rupesh01",
"email": "rupesh@gmail.com",
"role": "Owner"
},

"createdBy": {
  "_id": "68a10001",
  "name": "Akash",
  "userId": "akash01",
  "email": "akash@gmail.com",
  "role": "Manager"
},

"updatedBy": {
  "_id": "68a10003",
  "name": "Yaswanth",
  "userId": "yaswanth01",
  "email": "yaswanth@gmail.com",
  "role": "Developer"
}
}
```

The database should **still store only the IDs**.

You are only changing how the data is returned through the API.

6. Task Response — Expected Result

Currently, a Task might return:

```
{  
  "taskName": "Implement Login API",  
  "taskId": "0001",  
  "projectId": "68b20001",  
  "userId": "68a10001"  
}
```

You need to return the related Project and User information.

For example:

```
{  
  "taskName": "Implement Login API",  
  "taskId": "0001",  
  "projectId": {  
    "_id": "68b20001",  
    "projectName": "EstiMate-Pro",  
    "projectId": "0001"  
  },  
  "userId": {  
    "_id": "68a10001",  
    "name": "Akash",  
    "userId": "akash01",  
    "email": "akash@gmail.com",  
    "role": "Developer"  
  }  
}
```

Again, the database should continue storing:

Task

|

|— projectId → Project _id

|— userId → User _id

Only the API response should contain the related information.

7. How Can You Implement This?

You can choose **any appropriate MongoDB/Mongoose approach.**

Option 1 — Mongoose populate()

Option 2 — MongoDB Aggregation

Note: Above options are just for reference. You may also use any other related applicable approaches if you want.

8. Important Security Requirement

Do **not** return sensitive User information.

For example, the User collection contains:

```
{
  "name": "Akash",
  "email": "akash@gmail.com",
  "password": "$2b$12$....."
}
```

The password **must never appear** in the Project or Task response.

Only return appropriate fields such as:

```
{
```

```
"_id": "68a10001",  
"name": "Akash",  
"userId": "akash01",  
"email": "akash@gmail.com",  
"role": "Developer"  
}
```

Do not expose:

password

or any other sensitive information.

9. Apply This to CRUD Operations

The populated/expanded response should be considered for all relevant Project and Task operations.

Projects

Implement the relationship data in responses for:

GET /projects

GET /projects/:id

POST /projects

PUT /projects/:id

DELETE /projects/:id

Tasks

Implement the relationship data in responses for:

GET /tasks

GET /tasks/:id

POST /tasks

PUT /tasks/:id

DELETE /tasks/:id

For **create/update**, the request should still contain IDs.

For example:

```
{  
  "projectName": "EstiMate-Pro",  
  "managerId": "68a10001",  
  "ownerId": "68a10002"  
}
```

You should not send the entire User object while creating a Project.

The server uses those IDs to find the related records and returns the populated data in the response.